

HelixCode

HelixCode

İş dağıtan, koruyan ve asla kaybetmeyen AI tabanlı dağıtık geliştirme platformu.

Özet

HelixCode, kurumsal düzeyde, Go tabanlı dağıtık bir AI geliştirme platformudur. Geliştirme işlerini, SSH tarafından yönetilen çalışanlar ağı üzerinde akıllı bir şekilde bölümlere ayırır, otomatik kontrol noktaları ve geri alma işlevleriyle ilerlemeyi korur ve hiçbir işin kaybolmamasını sağlar. Çoklu LLM sağlayıcı entegrasyonunu, tam geliştirme yaşam döngüsü iş akışlarını ve platformlar arası teslimatı REST, CLI, TUI ve MCP arayüzleri altında birleştirir.

Kısa Açıklama

HelixCode, Go ile yazılmış dağıtık bir AI geliştirme platformudur. İşleri, SSH tabanlı çalışan ağları üzerinde akıllı görevlere böler, otomatik kontrol noktaları ve geri alma ile ilerlemeyi korur, birden fazla LLM sağlayıcısını entegre eder ve REST, CLI, TUI ve MCP arayüzleri aracılığıyla tam geliştirme yaşam döngüsünü yönetir.

Uzun Açıklama

HelixCode, kurumsal düzeyde dağıtık bir AI geliştirme platformudur (dev.helix.code, MIT) ve sloganıyla verdiği basit vaadi kelimesi kelimesine yerine getirir: işi böl, koru ve asla yerini kaybetme. Akıllı görev bölümü, otomatik iş koruması ve platformlar arası geliştirme iş akışları için tasarlanmış olup, dağıtık bilgi işlem gereksinimleri için Go ile yazılmıştır. Otomatik kontrol noktaları, geri alma ve gerçek zamanlı izleme, isteğe bağlı eklentiler değil, temel öncelikler olarak sunulur.

Mimarisinde, REST + WebSocket + MCP API yüzeyi altında odaklanmış çekirdek hizmetler yer alır: JWT kimlik doğrulama ve oturum yönetimi, sağlık izleme özelliğine sahip SSH tabanlı çalışan havuzu yönetimi, kontrol noktaları ve bağımlılık yönetimi içeren görev yönetimi, proje ve iş akışı yönetimi ile birleşik LLM sağlayıcı katmanı. Tüm bunlar PostgreSQL üzerinde kalıcı hale getirilirken, Redis isteğe bağlı koordinasyon ve önbellek katmanı olarak sunulur. Dağıtık çalışanlar ağ üzerinden otomatik olarak kurulur; bu sayede sistemin ölçeklendirilmesi, sunucunun bir makineye yönlendirilmesi kadar basit hale gelir. Çoklu istemci arayüzleri CLI, terminal kullanıcı arayüzü, REST ve mobil çerçeveleri kapsar; böylece aynı platform bir betikten, terminalden veya bir uygulamadan erişilebilir.

HelixCode, baştan sona eksiksiz bir geliştirme yaşam döngüsünü yönetir: planlama, oluşturma, test etme ve yeniden yapılandırma iş akışları, bağımlılık farkındalığı ve çoklu oturum bağlamı takibiyle otomatik olarak yürütülür. Böylece uzun süren bir çalışma, kesintiler ve makine sınırları arasında bile akışını korur. Birden fazla LLM sağlayıcısını (llama.cpp, Ollama ve OpenAI) tek bir arayüz altında birleştirir, mevcut CPU/GPU/belleği algılayarak modele uygun donanım seçimi yapar ve zincirleme düşünme ve ağaç tabanlı düşünme gibi ileri akıl yürütme stratejilerini, tek seferlik çözümlerin yetersiz kaldığı sorunlar için destekler. Model Context Protocol, standartlaştırılmış araç ve bağlam alışverişi için birden fazla iletim protokolü üzerinden uygulanır. Dağıtık çalışmalar ilerledikçe Slack, Discord, e-posta ve Telegram üzerinden çok kanallı bildirimlerle ekipler bilgilendirilir. Platform, Linux, macOS, Windows, Aurora OS ve SymphonyOS'u hedefler.

Neden inşa ettik

Dağıtık ve AI destekli geliştirme, görevler makineler arasında bölündüğünde ya da kesintiye uğradığında genellikle bağlam ve ilerleme kaybına yol açar. HelixCode, görev bölmeyi akıllı hale getirmek ve çalışma korumasını otomatikleştirmek için inşa edildi — böylece büyük bir geliştirme çabası parçalara ayrılabilir, bir çalışan ağına dağıtılabilir, kontrol noktaları alınabilir ve durum kaybı olmadan devam ettirilebilir ya da geri alınabilir.

Neden oyunun kurallarını değiştiriyor

Dağıtık AI geliştirmeyi *kalıcı* hale getiriyor — bu, ekiplerin bu parçaları elle bir araya getirirken asla pratik olmayan bir yetenek. Normalde üç ayrı araçta bulunan üç şey tek bir platformda birleşiyor: dağıtık hesaplama (otomatik kurulum ve sağlık izleme özellikli SSH çalışan ağları), AI geliştirme desteği (akıl yürütme ve araç çağırma özellikli çoklu sağlayıcı LLMs) ve tam yaşam döngüsü iş akışı otomasyonu. Bağlayıcı doku, veritabanı destekli

kontrol noktaları: Görev durumu, kontrol noktaları ve bağımlılıklar PostgreSQL'da saklandığından, birçok makine ve oturumu kapsayan bir iş, tam olarak kaldığı yerden geri alınabilir ya da devam ettirilebilir. Kesintiler ve bölünmüş çalışmalar, ilerleme kaybının kaynağı olmaktan çıkıp rutin, kurtarılabilir olaylara dönüşüyor.

Yenilikçi olan ne

- **Çalışma koruması temel bir ilke olarak:** Dağıtık geliştirme görevlerine otomatik kontrol noktaları ve geri alma uygulanması, böylece ilerleme kesintiye ya da makine arızasına uğradığında yok olacağına hayatta kalıyor.
- **Donanım duyarlı model seçimi:** Algılanan CPU/GPU/belleği inceleyerek her görevi makinenin gerçekten iyi çalıştırabileceği bir modelle eşleştiriyor — çalışan başına manuel ayarlama gerektirmiyor.
- **Tek platform, beş giriş noktası:** REST, WebSocket, CLI, TUI ve MCP; MCP'un kendisi de birden fazla iletim yöntemi üzerinden sunuluyor, böylece araçlar ve araçlar nasıl bağlanırlarsa bağlansınlar entegre olabiliyor.
- **Platformlar arası erişim:** Genellikle masaüstü üçlüsüyle sınırlı kalmayıp Aurora OS ve SymphonyOS'u da kapsayarak, çoğu aracın göz ardı ettiği platformlara kadar çalışan filosunu genişletiyor.

En büyük teknik zorluklar ve çözümlerimiz

- **Dağıtık, kesintiye açık görevlerde çalışmanın kaybolmaması.** Bir iş makineler arasında bölündüğünde, herhangi bir çökme ya da kesinti normalde devam eden işi yarıda bırakır. Biz görevi kendisini kontrol noktaları ve bağımlılıkların taşıyıcısı olarak modelledik, PostgreSQL'da sakladık; böylece sistem, son iyi duruma geri dönebilir ya da oradan devam edebilir — dayanıklılık, kırılabilir bellek içi durumda değil, veri katmanında yaşıyor.
- **Heterojen bir çalışan filosunun yönetimi.** Linux, macOS, Windows, Aurora ve SymphonyOS makinelerinden oluşan bir ağ, kullanılabilirlik ve kurulum açısından sürekli değişen bir hedef. Bunu, SSH tabanlı kayıt, yeni düğümlere otomatik kurulum ve sürekli sağlık izleme yapan özel bir çalışan havuzu hizmetiyle çözüyoruz; böylece makineler gelip giderken filo biliniyor ve kontrol altında kalıyor.
- **Sağlayıcı ve donanım heterojenliği.** LLM arka uçları ve bunları çalıştıran makineler yetenek açısından büyük farklılıklar gösteriyor. Bunu, birleşik bir LLM sağlayıcı arayüzü arkasına gizledik ve bunu, donanım tespiti (CPU/GPU/bellek)

ile eşleştirdik; bu da akıllı model seçimini yönlendiriyor, böylece doğru model doğru makineye iniyor ve çağırın tarafın her ikisi hakkında da akıl yürütmesine gerek kalmıyor.

İçerik

Teknoloji Yığını

- **Go (1.26+ iç modül)** — Dağıtık işçi sistemi için tam olarak ihtiyaç duyulan özelliklere sahip olduğu için seçildi: orkestrasyon için ucuz paralellik sağlayan goroutine tabanlı eşzamanlılık ve herhangi bir düğüme otomatik olarak yüklenen, bağımsız çalışabilen bir ikili dosya. Tüm çekirdek hizmetleri ve CLI/sunucu ikili dosyalarını içerir.
- **Gin (HTTP çatısı)** — Düşük ek yük ile hızlı ve minimal bir REST katmanı sunması nedeniyle seçildi; tüm istemcilerin bağlandığı /api/v1 yüzeyini (kimlik doğrulama, işçiler, görevler, projeler) sunar.
- **PostgreSQL 15+ (pgx/v5 üzerinden)** — Dayanıklı kayıt sistemi olarak seçildi çünkü kontrol noktaları ve geri alma işlemleri için işlemsel kalıcılık gereklidir; dağıtık hesaplama şemasının 11 tablosunu (kullanıcılar, işçiler, görevler, projeler, oturumlar, ysa_sağlayıcıları, bildirimler) barındırarak işlerin korunmasını sağlar.
- **Redis 7+ (isteğe bağlı, go-redis/v9)** — Zorunlu bir bağımlılık haline gelmeden sıcak yolları hızlandıran isteğe bağlı bir önbellek ve koordinasyon katmanı olarak seçildi; bu sayede minimal bir kurulum yalnızca PostgreSQL ile çalışabilir.
- **SSH** — İşçi kontrol iletişim kanalı olarak seçildi çünkü zaten her yerde mevcut ve güvenli; özel bir aracıya ihtiyaç duymadan tüm havuzda işçi kaydı, otomatik kurulum ve uzaktan komut yürütme işlemlerini yönetir.
- **Model Context Protocol (MCP)** — Dış araçlar ve araçlarla standartlaştırılmış araç ve bağlam alışverişi için seçildi; istemcilerin bağlandığı her ortamda çalışabilen çoklu iletişim desteği ile uygulandı.
- **LLM sağlayıcıları (llama.cpp, Ollama, OpenAI)** — Yerel ve barındırılan çıkarım işlemlerini tek bir birleşik arayüz altında toplamak için seçildi; böylece donanım duyarlı seçim, bir görevi çağırının farkında olmadan yerel veya barındırılan bir modele yönlendirebilir.

Durum ve Dürüstlük Notları

- **Durum: beta.** README dosyası “TAMAMLANMIŞ / tüm 5 aşama” durumunu bildiriyor; bu tamamlılık proje tarafından beyan edilen bir durum olup bağımsız olarak doğrulanmadığı için sayfa bunu beta olarak değerlendiriyor.

- Yukarıdaki tüm ayrıntılar, depo README dosyasından alınmıştır; pazarlama ifadeleri (sloganlar) kaynak metriklerden ziyade editöryeldir.

Öncelik katmanı: Helix-birincil.